

Introduction to Machine Learning (SS 2023)

Programming Project

Author 1

Last name: Bekhtari
First name: Salma
Matrikel Nr.: 11904872

Author 2

Last name: Markov
First name: Nikolay Ivaylov
Matrikel Nr.: 12116549

I. INTRODUCTION

For this report we consider the task of predicting the power consumption based on the weather. We classify our machine learning problem as a regression problem.. We take data of temperature, humidity for bedrock and the atmosphere and other factors that can influence the power consumption like wind and attempt to predict as close as possible the power consumption.

The data-set itself is large with 65 different categories (or columns) with the final one being the power consumption of the area and we have 39997 different samples (or rows).

II. IMPLEMENTATION / ML PROCESS

A. Data, Methods and Method choices

Based on the problem we considered a few different methods that best fit the data-set and our situation. For our first method we decided on *multi-linear regression* because of the many variables with quasi independent relationships between each other and not highly correlated to each other. For our second method we decided on *random forest regression*. Both methods are good predicting methods and since we are doing predictions we saw that as a good fit. We did not see a reason to pre-process the data and did not do it.

Multi-linear Regression (MR) belongs to the family of Linear Regression algorithms. MR assumes a linear relationship between the variables (features) and the dependent variable (target). It models the relationship using a linear equation with the multiple variables. The goal is to find the best-fit line that minimizes the sum of squared differences between the predicted and actual values. It can handle multiple numerical and categorical features and is based on minimizing the least squares error. **Random Forest Regression (RFR)** belongs to the family of Ensemble Learning algorithms and is an extension of the Decision Tree algorithm. RFR combines multiple decision trees to make the predictions. Each tree is built on a random subset of the data and a random subset of features. The final prediction is the average of the predictions from individual trees. Random Forest Regression reduces overfitting compared to a single decision tree and provides better generalization. The individual tree consists of nodes and branches, where each node

represents a decision or a feature, and branches represent the possible outcomes or splits. The depth, number of nodes, and branching criteria can be specified. The architecture is determined during the training process based on the data and hyperparameter settings.

The multi-linear regression was decided over because of its simplicity and it is easy to interpret and work with. As we said previously we consider the data to be independent to each other. The only disadvantage this method has over others is that the data-set itself is quite large and possibly has outliers that will affect the end result too much. For the choice of our second method we decided for its good handling of large data-sets. Between RFR and simple decision trees we asked ourselves which would be better suited and we decided on RFR as decision trees are more prone to overfit the data and might be unstable if there is a big difference in the samples. RFR did prove that it was slow and computational heavy for jupyter and we had to handicap it to use less trees and reduce the max-dept to be able to use it.

B. Hyperparameters

For the **Multivariate Linear Regressor**, the weights or coefficients of the regression model were calculated using the pseudo-inverse formula. This involved separating the feature values, adding a column of 1s as the intercept term, and solving the system of linear equations. The resulting weights were then used to predict the power consumption based on new input values. No other hyperparameters were used.

For the **Random Forrest Regressor** we have number of trees and the max depth of trees as our parameters. Due to the constrains of jupyter and slow computation time we settled on 10 for the number of trees and 5 for the max depth. We conducted tests on jupyter to maximize score but at the same time to not take an entire evening to train the model to the predictions. We also choose to use MSE instead of RMSE for what he hoped for more accurate results. The results and tests will be mentioned below.

III. RESULTS

The performance of regression models is often evaluated using various metrics that quantify the predictive accuracy of

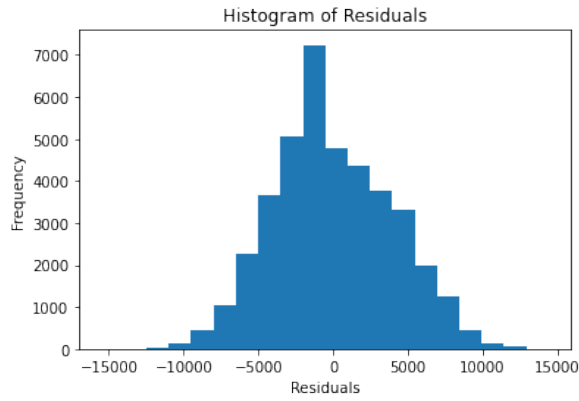


Fig. 1. Residuals of Multivariate Linear Regression Model

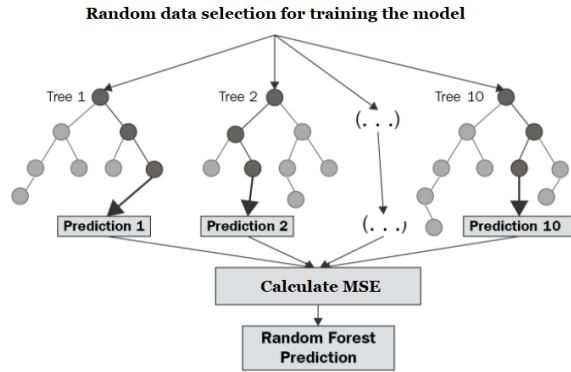


Fig. 2. Architecture of Random Forest Regression (10 trees, max depth 5)

the model. In our case, we focus on the mean absolute error (MAE), which is a widely used metric in regression tasks. MAE calculates the average absolute difference between the actual and predicted values, also known as the residual. To calculate the MAE, we split the data set into training data and a subset used to compare the predictions with the actual data. After making the calculation, we get MAE 3233.17. A lower MAE indicates a lower mean error rate. It is worth noting that the MAE of our model is lower than that of `sklearn.dummy.DummyRegressor()`, which is 4085.99. An important regression modeling is that the errors are independent and normally distributed. This means that some degree of error is to be expected due to the inherent uncertainties in forecasting. Thus, the regression model can be expressed as the sum of the deterministic component and the stochastic (random) component: $\text{response} = \text{deterministic} + \text{random}$.

Ideally, our model would include deterministic (predictive) information in this equation. This means that if the residuals are random and unpredictable, they should follow a normal distribution. To assess this criterion, we constructed histograms of all residuals, and Figure 1 shows that they fit a normal curve. A normal distribution of residuals indicates a certain degree of normality in the data generation process, which is a desirable property of a regression model.

The results for the multi-linear regression method model

are [Train Data-set Score:233.180] and [Test Data-set Score: 3271.442]. The results using Random Forrest Regression method model are [Train Data-set Score: 4447.672] and [Test Dataset Score: 4345.987]. From the results we can tell that the multi-linear solution works better. It was also easier to code, much more simpler and training time is significantly less to no surprise. With RFR we conducted tests with different hyperparameters to see if the prediction and training improves, we also tried to keep the training and prediction times low and not be extremely long. On the code we have commented the different tests and results with exception of those that took over 20 minutes, those were cancelled and have no results. In general the results did not improve significantly to justify their wait time.

IV. DISCUSSION

The RFR method has a poor performance and is overly complicated. What could be done to improve the method is reshaping the data before the final prediction. Additionally data pre-processing can also contribute to a better prediction and better training of the model. The model can also be trained with more trees and a higher max depth. But a general improvement could be re-coding the entire 2 classes as they could be made more simple and better functioning. The multi-linear method performed better most likely because it is much simpler and is more straight forward. For what went wrong (we don't know if our score is good but we assume it is mediocre) the same thing can be said as RFR, we can pre-process the data for better training of the model.

We briefly tried reshaping and pre-processing the data for RFR but ultimately it didnt improve training and prediction and we were limited on time as the end date was coming. For the ML method we tried implementing regularization techniques like L1 and L2 but ended up removing as we couldnt set them up properly.

For improving our results for ML we would definitely revisit a regularization technique like L1. We believe that would be the easiest improvement to the code that can be implemented and will most likely yield significantly better results with a bit of tinkering of the lambda. For RFR we are convinced that reshaping the data can better the final result if done properly and with more time we would improve that first. Secondly perhaps swapping from mse to rmse would be better if the current predictions deviate because the model is too much influenced by outliers in the data. Both methods can benefit from pre-processing data as well.

V. CONCLUSION

The main takeaway from this project for us has been is simpler code is much easier to optimize and work with. We wasted a lot of time on writing RFR and after that removing errors. If we were to spend the same time on ML regularization it would have been time better spent. If we were to start the project again we would most likely pick the ML method again but change to Decision trees for our second method. Additionally we could have spent more time planning the architecture and not make it as we go.